大模型安全情报采集
智能体软件 V0.1.0

用户使用手册

1 概述

1.1 关于本手册

本手册说明大模型安全情报采集智能体软件的运行环境、配置方法以及全量采集、增量采集、结果查询、追踪回放和技巧库管理的使用步骤。软件面向大模型与人工智能安全领域，集中采集公开漏洞信息和研究文献，形成可以查询、导出和持续更新的情报记录。

本手册适用于大模型安全情报采集智能体软件 V0.1.0。V0.1.0 为本系统的软件版本号，与 Python 运行环境版本无关；Python 版本要求见“本地部署与安装”章节。用户通过 Gradio 浏览器控制台或 intel-agent 命令行执行任务。控制台与命令行调用同一采集核心，共享所配置的历史存储和技巧库。

1.2 使用对象

适用于安全研究人员、情报分析人员及负责软件运行维护的人员。日常用户需要理解采集主题、时间范围和调用预算；部署人员需要能够安装 Python 环境、编辑配置文件并启动本地服务。

1.3 平台介绍

系统接入 NVD 漏洞接口、arXiv 文献接口、CISA 已知被利用漏洞目录和 OSV 开源软件漏洞接口。通过话题识别、相关性判断、记录合并和覆盖分析，将不同来源的结果整理为统一情报条目。具备模型密钥时，Claude Agent SDK 驱动模型调用项目提供的搜索工具；缺少模型配置或会话失败时，可降级使用规则采集。

2 本地部署与安装

2.1 部署准备

本地部署适合单机采集，默认使用 JSON 文件，若安装并配置了 MongoDB 环境可以替代 JSON 文件存储。项目根目录下，backend/ 存放后端代码，其中 backend/intel_agent/ 为情报采集核心，backend/console/ 为可视化控制台。pyproject.toml 定义项目依赖和启动入口，uv.lock 锁定依赖版本，.env.example 提供环境变量配置示例，复制为 .env 后填写实际配置。这些文件共同用于配置项目运行环境；本手册中的命令均在项目根目录执行。

运行环境为 Python 3.10 至 3.13，建议使用 Python 3.12；需要 uv 包管理工具及可访问软件依赖仓库、公开情报源的网络。智能采集还需能够访问所配置的模型接口。浏览器使用现代 Chrome、Edge 或 Safari。

2.2 第一步 安装依赖

在终端进入解压后的 LLM-Intel-Auto 目录。先执行 python --version 和 uv --version 检查工具。若未安装 uv，可在已有 Python 环境执行 python -m pip install uv，安装后重新打开终端。

执行以下命令，按项目锁文件安装依赖并创建虚拟环境：

uv sync --locked --group dev

该命令包含运行依赖与测试依赖。安装结束且未报错后继续配置；不需要手工激活虚拟环境，后续统一通过 uv run 启动。

2.3 第二步 创建配置文件

首次部署将根目录.env.example 复制为.env。可保持模型密钥为空，需要使用大模型能力自动化检索时，按下一章填写供应商和密钥。默认本地配置为 GRADIO_SERVER_NAME=127.0.0.1、GRADIO_SERVER_PORT=8000；默认可以不配置 INTEL_MONGO_URI。

3 首次启动与部署验证

3.1 第三步 启动控制台

在项目根目录执行:

uv run web

保持终端运行，在浏览器打开 http://127.0.0.1:8000/ 。若已修改 GRADIO_SERVER_PORT，以启动终端实际显示地址为准。打开后应看到 Collect、情报态势、Trace replay、Database 和 Memory 五个页签。

首页还显示采集配置、主模型和快速模型。规则采集可以访问真实公开数据源；SDK 智能采集表示本地模型配置检查通过，实际是否使用模型仍需查看任务追踪。界面不需要注册或登录。

3.2 第四步 执行首次验证

进入 Collect，Mode 选 full，Max rounds 填 1，Max API calls 填 4，保留 Verbose 勾选，然后点击 Run collection。等待 Run summary 和 Collected items 更新，记录 run_id。

查看 Agent verbose trace 是否出现工具调用、来源错误或 [fallback]。出现[fallback]表示模型会话降级为规则采集；任务完成不等同于所有来源均成功，也不保证本次一定获得新条目。

点击 Load latest run，确认刚完成的记录可以再次加载。再进入 Database，点击 Refresh store status；默认应使用 JSON 存储。也可执行 uv run intel-agent latest 核对最近持久化结果。

3.3 停止 重启与配置生效

停止本地服务时，在启动终端按 Ctrl+C；再次执行 uv run web 可重启。修改.env 后应停止并重新启动服务。尽量等待采集完成后再停止，以便任务正常保存。

端口占用时，将.env 中 GRADIO_SERVER_PORT 改为其他可用端口，例如 8001，再重启并访问对应地址。服务默认仅允许本机访问；当前界面没有账户认证，不应直接作为公共互联网服务开放。

4 Docker 与 MongoDB 部署

4.1 完整容器部署

此方式适合已安装并启动 Docker Engine 或 Docker Desktop、且可使用 docker compose 的环境。先在项目根目录准备.env 并按需填写模型配置，再执行：

docker compose up -d --build web

Compose 会构建控制台镜像，同时启动并等待 MongoDB 健康检查通过。执行 docker compose ps 检查 web 与 mongo 状态，执行 docker compose logs --tail=100 web 查看启动日志，随后打开 http://127.0.0.1:8000/。

Compose 固定将宿主机 8000 映射到容器 8000，并覆盖容器内监听地址和端口。需要更换外部端口时，编辑 docker-compose.yml 的 web.ports，例如改为 8001:8000；仅修改.env 中端口不会改变该映射。

4.2 仅使用容器数据库

若控制台仍在本地运行，只执行 docker compose up -d mongo，然后在本地.env 设置 INTEL_MONGO_URI=mongodb://localhost:27018，重新执行 uv run web。数据库名可通过 INTEL_MONGO_DB 指定，默认 intel_agent。

完整容器部署在容器内部使用 mongodb://mongo:27017；本地程序通过宿主机发布端口连接。若修改 INTEL_MONGO_HOST_PORT，本地连接地址也应同步修改。切换存储后看到的是所选存储中的记录，已有 JSON 数据不会自动迁移到 MongoDB。

4.3 运行维护与数据位置

修改容器使用的.env 后，执行 docker compose up -d --force-recreate web 重新创建服务。停止容器可执行 docker compose stop；重新启动执行 docker compose up -d web。

批量采集可执行 docker compose run --rm --build intel-agent full --max-rounds 1 --max-api-calls 4 --verbose。运行记录存入 MongoDB；技巧库、缓存及追踪通过./data 挂载保留。MongoDB 数据使用 mongo_data 命名卷，备份时需单独备份数据库，不要使用带 -v 的删除命令清除数据卷。

5 智能采集配置示例

5.1 使用 DeepSeek 采集

在根目录 .env 中填写以下配置；密钥值替换为自己的实际密钥，其他行可直接按示例填写。模型名称采用当前项目代码默认值，若所用接口不再支持该名称，应填写该接口实际可用的模型标识。

```
INTEL_SDK_PROVIDER=deepseek
DEEPSEEK_API_KEY=在此填写自己的密钥
INTEL_SDK_MODEL=deepseek-flash
INTEL_SDK_FAST_MODEL=deepseek-flash
GRADIO_SERVER_NAME=127.0.0.1
GRADIO_SERVER_PORT=8000
```

主模型负责采集会话；快速模型用于代码中选择快速模型的辅助决策。它们分别由 INTEL_SDK_MODEL 和 INTEL_SDK_FAST_MODEL 指定，也可以设置为同一模型。

5.2 确认配置已生效

保存 .env 后按部署方式重启服务，再检查首页显示的供应商相关状态及两个模型名称。运行一次小预算采集，查看 Agent verbose trace；只有实际调用过程才能确认远程接口是否成功。

若首页仍显示旧配置，检查启动进程的环境变量是否覆盖 .env，并确认终端位于本项目根目录。若配置了 ANTHROPIC_BASE_URL 和 ANTHROPIC_AUTH_TOKEN，它们会覆盖供应商端点与密钥；直接使用供应商接口时应清除无意保留的自定义覆盖。

5.3 规则采集与可选增强

模型密钥为空时，可用规则方式体验数据源采集。只配置 DeepSeek 即可启用模型采集，不必同时填写 GLM；GLM_API_KEY 对应的语义过滤与 embedding 属于可选增强。

密钥只写入配置文件，不写进采集主题或导出材料。NVD_API_KEY 为可选数据源配置；不填写时仍可请求数据，但可能受到更严格的限流。

6 采集配置

6.1 模型接口设置

在根目录 .env 中设置 INTEL_SDK_PROVIDER=deepseek，并填写 DEEPSEEK_API_KEY。当前 DeepSeek 主模型和快速模型默认均为 deepseek-flash；需要显式指定时，分别设置 INTEL_SDK_MODEL 和 INTEL_SDK_FAST_MODEL。密钥只填写在本地配置中，不应粘贴到采集主题、日志说明或导出文件中。

DeepSeek 使用 https://api.deepseek.com/anthropic 兼容端点。选择 glm 时填写 GLM_API_KEY。自定义兼容接口可同时配置 ANTHROPIC_BASE_URL 和 ANTHROPIC_AUTH_TOKEN，这两项优先于供应商的端点和密钥。

6.2 可选增强与存储配置

NVD_API_KEY 为可选数据源密钥。GLM_API_KEY 还可用于 GLM embedding 与相关性复核；仅填写 DeepSeek 密钥时，模型采集可以运行，但不会自动启用 GLM 语义过滤。相关性过滤、技巧库及其语义召回可分别通过 INTEL_RELEVANCE_ENABLED、INTEL_PLAYBOOK_ENABLED、INTEL_PLAYBOOK_EMBED 调整。

默认结果保存为本地 JSON。设置 INTEL_MONGO_URI 后，程序优先使用 MongoDB；可通过 INTEL_MONGO_DB 指定数据库名。配置 MongoDB 不可达时应修复连接或移除该设置，不要把数据库连接失败理解为自动切回 JSON。

6.3 配置生效与状态确认

配置按系统环境变量、根目录 .env、代码默认值的顺序解析；上级启动环境已设置同名变量时会覆盖文件中的值。修改配置后重新启动 Gradio 服务。

首页显示采集配置、主模型和快速模型。“SDK 智能采集”表示配置检查通过，不代表每次远程调用都成功。实际执行情况应结合 Agent verbose trace 判断；出现 [fallback] 表示该轮模型会话已降级。

7 平台首页与任务参数

7.1 功能入口

首页包含 Collect、情报态势、Trace replay、Database 和 Memory 五个页签。Collect 用于创建任务并查看即时结果；情报态势显示历史语料的覆盖和来源情况；Trace replay 用于读取运行日志；Database 用于查询、导出情报；Memory 用于查看技巧库与相关性缓存。

7.2 采集参数

Mode 选择 full 或 incremental。full 为全量采集；incremental 为增量采集。Incremental focus 为增量关注主题或实体，例如 prompt injection；Window days 为首次建立时间范围时的回看天数。full 模式不使用这两个增量参数。

Max rounds 控制任务最多执行的轮次，控制台默认 4；Max API calls 控制源搜索 API 调用预算，控制台默认 20。该预算不等同于模型调用次数或总费用上限。建议首次使用设置 1 轮、2 至 4 次源调用，确认接口和结果后再扩大预算。

Verbose (live agent trace) 控制是否向界面实时输出追踪，默认勾选。Run collection 启动采集；Load latest run 读取已完成并持久化的最近一次运行，不会发起新的搜索。

7.3 结果区域

Run summary 显示 run_id、run_mode、轮次、情报数量、源调用数、覆盖缺口及终止原因等字段。Collected items 显示条目标识、来源、标题、相关性、话题、发布时间和来源地址。

界面显示任务完成不表示全部话题均已覆盖。用户应同时查看 open_gaps、stop_reason 和追踪日志，区分预算结束、低收益停止、模型降级和数据源错误。

8 全量情报采集

8.1 发起全量任务

第一步，进入 Collect 页签，将 Mode 设置为 full。第二步，设置 Max rounds 与 Max API calls，勾选 Verbose。第三步，单击 Run collection。系统创建运行标识并根据大模型安全话题组织搜索。

模型会在允许的工具范围内查询公开来源、观察返回结果、记录检索技巧并提交轮次总结。确定性控制层负责限制预算、合并条目、更新覆盖缺口，并根据轮数、预算和收益决定是否继续。

8.2 覆盖范围与判定

核心话题包括提示注入、越狱、智能体工具滥用、数据泄露、模型供应链、RAG 投毒等。代码定义 18 个 Core 话题和 15 个 Extended 话题。全量配额以 Core 话题为目标，Extended 可被检索与标注，但不计入核心配额。

INTEL_COVERAGE_QUOTA 默认值为 3，用于全量模式的相关情报配额判断。覆盖状态来自系统当前检索结果及规则，不表示已经穷尽互联网上的全部安全事件。相关性分数和话题标签用于筛选，需要分析人员进一步核查原始来源。

8.3 查看和复核结果

采集结束后记录 run_id，检查结果表及开放缺口。点击或复制来源地址进入原始资料，核对标题、漏洞编号、发布时间和描述。需要补充覆盖时，可提高下一次任务预算，或切换增量模式围绕具体主题检索。

命令行等效示例：uv run intel-agent full --max-rounds 1 --max-api-calls 4 --verbose。全量模式在结果中对应 run_mode=bootstrap。多次执行会生成独立运行记录，不会清空历史库。

9 增量情报采集

9.1 发起增量任务

进入 Collect，将 Mode 设置为 incremental，在 Incremental focus 填写关注主题或实体，如 prompt injection、agent tool abuse；设置 Window days、最大轮次和调用预算后单击 Run collection。关注字段留空时按增量时间范围执行。

默认使用 adaptive 策略。系统根据历史语料的主题缺口提出和审批候选查询，结合查询收益、噪声和重复情况调整检索选择。OSV 用于软件包或实体定向确认，不作为统一时间订阅源。

9.2 时间范围与历史水位

NVD 和 arXiv 分别维护来源 checkpoint。默认对历史水位重叠回看 72 小时，以覆盖延迟发布或更新的记录；重叠时间通过 INTEL_INCREMENTAL_OVERLAP_HOURS 调整。已有历史水位时，实际起点可能按来源水位确定，而非简单采用界面天数。

命令行可用 --since 指定明确起点，例如：uv run intel-agent incremental --focus "prompt injection" --since 2026-09-01 --max-rounds 1 --max-api-calls 4 --verbose。日期未带时区时按 UTC 解析。默认窗口通过 --window-days 设置。

9.3 理解增量结果

增量任务可能返回零条新增：时间范围内无结果、候选结果已存在，或时间与相关性过滤后无新增均会造成这种情况。应结合来源返回数量、重复统计和日志判断，不应仅凭零新增认定服务异常。

新情报按条目标识合并，保留来源和话题等元数据。重复采集可更新历史关联信息，并用于后续覆盖分析与技巧收益统计。若需要采用旧增量策略，可设置 INTEL_INCREMENTAL_STRATEGY=legacy 后重启服务。

10 情报态势与追踪回放

10.1 查看情报态势

进入情报态势页签，单击“刷新统计”。系统从当前存储读取历史情报，展示核心指标、Core 主题覆盖矩阵、覆盖全面性与数据健康、开放覆盖缺口、核心话题情报分布及来源质量与采集效率。

页面还显示近 12 周有效情报趋势、Extended 主题趋势和待审核候选主题。核心覆盖按主题与所需证据通道计算，并考虑相关性、来源可信度、事件唯一性和时效；该指标与全量任务的条数配额不是同一统计口径。

Extended 主题不进入 Core 覆盖率分母。候选主题用于提示可能需要关注的新方向，当前页面提供展示，不包含把候选主题直接批准加入核心分类的操作。

10.2 实时追踪

Collect 的 Agent verbose trace 展示模型文本、工具调用、工具结果、轮次统计和会话结束信息。用户可据此观察实际使用的检索条件与预算消耗。模型文字属于过程说明，应以持久化记录和原始来源作为结果核查依据。

出现 [fallback] 时表示规则降级；出现 API-call budget exhausted 表示数据源调用额度已用完。预算耗尽并不意味着模型接口错误。模型会话提前结束时，系统可能保留已获得的情报，而非重新采集整轮。

10.3 回放历史日志

进入 Trace replay，输入完整 run_id，单击 Load trace JSONL。系统读取 data/intel_agent/traces 下的同名 JSONL 文件并展示追踪。找不到日志时检查运行标识与当前项目数据目录。

命令行执行任务时使用 --verbose 才会创建对应追踪输出。控制台采集会建立追踪文件；Verbose 主要影响界面是否实时展示事件。更换工作目录或未迁移追踪文件时，历史情报存在也可能无法回放日志。

11 历史查询与数据导出

11.1 运行历史

进入 Database，点击 Refresh store status 查看存储类型、运行数量和去重条目数量。在 Runs 区域设置 Limit 后点击 List runs。运行表显示 run_id、状态、保存时间、条目数、运行模式、轮次、目标、覆盖缺口和终止原因。

Download runs CSV 提供当前运行查询结果的 CSV 下载。查询不会重新执行采集，也不会自动删除历史任务。最近结果可通过 Collect 的 Load latest run 或命令 uv run intel-agent latest 查看。

11.2 情报条目检索

在 Items (global intel) 中按 Source、Topic、Text search、Filter by run_id 设置筛选条件。文本搜索匹配标题、摘要或条目标识；留空条件不限制对应字段。设置 Limit 后点击 Query items。

结果包含 item_id、来源、标题、摘要、相关性分数、话题、发布时间、来源地址及首次/最近关联运行等信息。数据源选项需要更新时点击 Refresh source list。可结合 run_id 筛选定位一次任务的结果，结合话题和关键词缩小范围。

11.3 导出与单次运行详情

查询完成后通过 Download items CSV 下载当前筛选结果。导出文件保存在 data/exports；Limit 会影响导出条目范围，需要更多记录时应调整查询上限后重新查询。CSV 可由表格软件打开，中文显示异常时选择 UTF-8 编码导入。

展开 Run detail，输入 run_id，点击 Load run + its items 查看该运行的摘要及条目。页面中的 engine 字段标识采集引擎模块，不足以单独判断某轮是否使用模型，SDK 或规则路径应从追踪中核实。

12 技巧库与存储维护

12.1 检索技巧库

进入 Memory，点击 Refresh memory status 查看技巧库和相关性缓存状态。在 Playbook (query strategies) 中按 Topic、Source filter、Status、Text search 筛选并设置 Limit，点击 Load playbook。

技巧记录包括 entry_id、来源、话题分组、收益、调用次数、命中情况、检索算子签名、技巧说明、创建运行及最近使用时间等。系统会根据实际采集收益收获技巧、召回已有策略并衰减低收益技巧。页面用于查看与导出，不提供任意手工编辑技巧的入口。

12.2 相关性缓存

在 Relevance embedding cache 中按 Topic、Min embedding score 和 Limit 筛选，点击 Load relevance cache 查看缓存摘要。缓存按标题和摘要的哈希保存评分、话题等信息。未启用在线 embedding 或未形成缓存时列表可为空，这不妨碍规则过滤和基础采集。

技巧库与缓存各自提供 CSV 下载。查看缓存不等于重新计算相关性；用户应结合采集时启用的过滤配置理解分数。

12.3 文件保存与备份

JSON 模式每次运行写入 data/intel_runs/<run_id>.json，latest.json 保存最近运行索引。技巧库默认位于 data/intel_agent/playbook.jsonl，相关性缓存位于 data/intel_agent/relevance_cache.jsonl，追踪文件位于 traces 子目录。

备份时应同时保留运行结果、技巧库和追踪文件。MongoDB 模式将运行和条目分别写入 runs 与 items 集合，条目以 item_id 去重；数据库另行备份。停止采集后再进行数据迁移，并保持新环境的模型配置与存储配置一致。

13 常见问题与操作示例

13.1 常见问题处理

网页无法访问：检查启动终端是否仍运行，确认端口、监听地址和当前项目路径。若打开旧副本，关闭旧网页并访问当前服务实际地址。

显示规则采集或 [fallback]：检查供应商与密钥设置是否正确、进程是否已重启、模型名称及兼容端点是否可用。配置检查通过不能保证余额、网络和远程服务始终正常，必要时查看对应运行追踪。

情报数量少或仍有缺口：检查预算是否过小、关注主题是否过窄、时间窗口或历史水位是否限制结果。网络限流、来源无匹配及重复条目也会降低新增量。不要将覆盖缺口等同于程序运行失败。

数据库读取失败：检查 INTEL_MONGO_URI、数据库服务和网络连接。使用 Compose 数据库时，容器内连接 mongo:27017，宿主机按发布端口连接，项目默认发布端口为 27018。

13.2 一次小预算采集示例

启动控制台后确认顶部显示 SDK 智能采集，主模型为 deepseek-flash。进入 Collect，选择 full，将 Max rounds 设为 1、Max API calls 设为 2，勾选 Verbose 后执行。观察工具调用和轮次总结，完成后记录 run_id。

进入 Database，按该 run_id 查询条目并下载 CSV；进入 Trace replay，输入相同 run_id 回放过程；进入情报态势刷新统计，查看新增记录对覆盖的影响。该流程将采集、复核、导出和追踪串联起来。

13.3 日常使用顺序

首次使用先完成小预算采集与来源复核，再扩大全量覆盖。后续围绕关注主题执行增量采集，定期查看开放缺口和来源质量，并备份运行结果。模型输出和来源内容均需结合安全研究任务判断，不直接替代人工研判。